CRM + KANBAN · DOKUMENTASI TEKNIS

Panduan Instalasi, GitHub & Deploy ke VPS (macOS)

Langkah demi langkah dari Mac (macOS) sampai aplikasi online di domain sendiri, dengan deploy otomatis setiap kali kode di-push ke GitHub.

macOS + Homebrew | Ubuntu 22.04 / 24.04 | Node.js 22 | PostgreSQL 16 | PM2 | Nginx | Let's Encrypt | GitHub Actions

Versi 31 Oktober 2026

Daftar isi

1. Gambaran besar

2. Yang perlu di-install di laptop (macOS)

3. Hubungkan proyek ke GitHub

4. Siapkan VPS Ubuntu

5. Software yang di-install di VPS

6. Ambil kode dari GitHub ke VPS

7. Konfigurasi `.env` produksi

8. Menjalankan pertama kali

9. Domain & HTTPS

10. Deploy otomatis dengan GitHub Actions

11. Perawatan

12. Troubleshooting

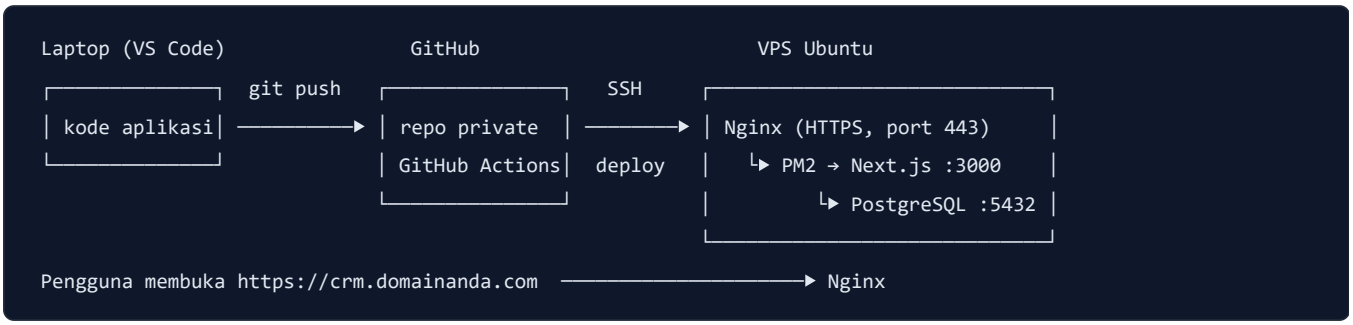
13. Checklist ringkas

Panduan ini membawa aplikasi CRM + Kanban dari laptop Anda sampai online di domain sendiri. Urutannya: siapkan laptop, simpan kode di GitHub, siapkan VPS, lalu aktifkan deploy otomatis. Setelah selesai, setiap kali Anda push ke GitHub, aplikasi di VPS ikut ter-update sendiri.

Yang dipakai: Ubuntu 22.04/24.04 · Node.js 22 · PostgreSQL 16 · PM2 · Nginx · Let's Encrypt (HTTPS gratis) · GitHub Actions

> **Cara membaca perintah:** baris yang diawali `%` dijalankan di aplikasi **Terminal** di Mac (buka dengan `Cmd+Spasi`, ketik "Terminal", Enter). Baris yang diawali `$` dijalankan di terminal VPS (lewat SSH). Ketik perintahnya saja, tanpa `%` atau `$`. Teks seperti `crm.domainanda.com`, `IP_VPS`, `USERNAME` harus diganti dengan milik Anda.

1. Gambaran besar



- **GitHub** menyimpan kode dan riwayat perubahan.
- **GitHub Actions** mengecek kode lalu menyuruh VPS meng-update dirinya.
- **Nginx** menerima pengunjung di domain Anda, mengurus HTTPS, lalu meneruskan ke aplikasi.
- **PM2** menjaga aplikasi tetap hidup (restart otomatis jika crash atau VPS reboot).
- **PostgreSQL** menyimpan semua data CRM.

2. Yang perlu di-install di laptop (macOS)

Berlaku untuk Mac Apple Silicon (M1/M2/M3/M4) maupun Intel, macOS 13 Ventura atau lebih baru.

Software	Untuk apa	Cara install
Xcode Command Line Tools	Berisi Git & alat dasar	Otomatis ikut saat install Homebrew
Homebrew	"App Store" untuk software developer	https://brew.sh
Node.js 22 LTS	Menjalankan & build aplikasi	https://nodejs.org (installer .pkg, pilih **LTS**)
VS Code	Editor kode	`brew install --cask visual-studio-code`
GitHub CLI	Login GitHub tanpa repot	`brew install gh`
Akun GitHub	Tempat menyimpan kode	https://github.com/signup

2.1 Homebrew

Salin perintah dari https://brew.sh ke Terminal (masukkan password Mac saat diminta):

TERMINAL · MAC

```
% /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Di Mac Apple Silicon, di akhir instalasi muncul **"Next steps"**. Jalankan dua perintah yang ditampilkan di sana, biasanya:

TERMINAL · MAC

```
% echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
% eval "$(/opt/homebrew/bin/brew shellenv)"
```

2.2 Node.js, VS Code, GitHub CLI

Untuk Node.js, unduh installer **LTS (.pkg)** dari https://nodejs.org lalu klik dua kali dan ikuti wizard. Cara ini paling sederhana dan versinya pasti 22.x. Lalu install sisanya lewat Homebrew:

TERMINAL · MAC

```
% brew install gh git
% brew install --cask visual-studio-code
```

Agar bisa membuka folder dari Terminal dengan `code .`: buka VS Code, tekan `Cmd+Shift+P`, lalu ketik **"Shell Command: Install 'code' command in PATH"**.

Tutup lalu buka lagi Terminal, kemudian cek:

TERMINAL · MAC

```
% node -v        # harus v22.x atau lebih baru (minimal v20.9)
% npm -v
% git --version
% gh --version
```

Atur identitas Git (cukup sekali seumur laptop):

TERMINAL · MAC

```
% git config --global user.name "Nama Anda"
% git config --global user.email "email-github-anda@contoh.com"
% git config --global init.defaultBranch main
```

Pastikan aplikasi sudah jalan di laptop dengan mengikuti README.md bagian "Menjalankan di komputer lokal".

3. Hubungkan proyek ke GitHub

3.1 Buat repository

1. Buka https://github.com/new
2. **Repository name:** `crm`
3. Pilih **Private** (wajib: ini aplikasi internal).
4. **Jangan** centang "Add a README", ".gitignore", atau "license" karena proyek sudah punya.
5. Klik **Create repository**.

3.2 Login GitHub dari laptop

Cara termudah dengan GitHub CLI:

TERMINAL · MAC

```
% gh auth login
```

Pilih: `GitHub.com` → `HTTPS` → `Yes` (authenticate Git) → `Login with a web browser`, lalu ikuti kode yang muncul di browser.

> Tanpa GitHub CLI juga bisa, tetapi GitHub tidak menerima password biasa untuk `git push`. Anda perlu membuat **Personal Access Token** (GitHub → Settings → Developer settings → Tokens) dan menempelnya saat diminta password. macOS menyimpannya di Keychain, jadi cukup sekali. GitHub CLI jauh lebih mudah.

3.3 Kirim kode pertama kali

Buka terminal di VS Code (**Terminal** → **New Terminal**) di folder proyek `crm`, atau di Terminal: `cd` ke folder proyek.

TERMINAL · MAC

```
% git init
% git add .
% git status
```

> **Perhatian, cek sebelum commit:** di daftar `git status` **tidak boleh** ada file `.env`, folder `node_modules`, atau `.next`. Semuanya sudah dikecualikan lewat `.gitignore`. Yang boleh ikut hanya `.env.example`. File `.env` berisi password database dan kunci rahasia, jadi jangan pernah di-upload.

TERMINAL · MAC

```
% git commit -m "Versi awal CRM + Kanban"
% git branch -M main
% git remote add origin https://github.com/USERNAME/crm.git
% git push -u origin main
```

Refresh halaman repo di GitHub; semua file sudah muncul.

3.4 Alur kerja sehari-hari

Setiap selesai mengubah kode:

TERMINAL · MAC

```
% git add .
% git commit -m "Jelaskan perubahan singkat"
% git push
```

Setelah bab 10 selesai, `git push` ini otomatis men-deploy ke VPS.

4. Siapkan VPS Ubuntu

4.1 Spesifikasi

	Minimal	Disarankan
CPU	1 vCPU	2 vCPU
RAM	1 GB (wajib swap, lihat 4.5)	2 GB atau lebih
Disk	20 GB SSD	40 GB SSD
OS	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS

Catat **IP publik** VPS dan **password root** dari penyedia VPS.

4.2 Login pertama kali

Dari Terminal di Mac (SSH sudah bawaan macOS):

TERMINAL · MAC

```
% ssh root@IP_VPS
```

Ketik `yes` saat ditanya fingerprint, lalu masukkan password root.

4.3 Update sistem & buat user `deploy`

Jangan menjalankan aplikasi sebagai root. Buat user khusus:

TERMINAL · VPS

```
$ apt update && apt upgrade -y
$ timedatectl set-timezone Asia/Jakarta
$ adduser deploy              # isi password, sisanya boleh Enter
$ usermod -aG sudo deploy
```

4.4 Login pakai SSH key (lebih aman dari password)

Di laptop, buat key (jika belum punya) lalu kirim ke VPS:

TERMINAL · MAC

```
% ssh-keygen -t ed25519 -C "laptop-saya"     # Enter terus (boleh isi passphrase)
% cat ~/.ssh/id_ed25519.pub | ssh root@IP_VPS "mkdir -p /home/deploy/.ssh && cat >> /home/deploy/.ssh/authorized_keys && chown -R deploy:deploy /home/deploy/.ssh && chmod 700 /home/deploy/.ssh && chmod 600 /home/deploy/.ssh/authorized_keys"
```

Jika Anda memberi passphrase, agar macOS mengingatnya di Keychain:

TERMINAL · MAC

```
% ssh-add --apple-use-keychain ~/.ssh/id_ed25519
```

Coba login sebagai deploy (seharusnya tanpa password VPS):

TERMINAL · MAC

```
% ssh deploy@IP_VPS
```

Jika berhasil, matikan login root & password. **Di VPS:**

TERMINAL · VPS

```
$ sudo nano /etc/ssh/sshd_config
```

Ubah/pastikan baris berikut (hapus tanda `#` di depannya):

```
PermitRootLogin no
PasswordAuthentication no
```

Simpan (`Control+O`, Enter, `Control+X`; pakai tombol **Control**, bukan Cmd), lalu:

TERMINAL · VPS

```
$ sudo systemctl restart ssh
```

> **Perhatian:** jangan tutup jendela SSH yang sedang terbuka. Buka jendela Terminal baru (`Cmd+N`) dan tes `ssh deploy@IP_VPS` dulu. Jika gagal, perbaiki dari jendela lama. Kalau keduanya tertutup, Anda harus masuk lewat konsol web penyedia VPS.

4.5 Firewall & swap

TERMINAL · VPS

```
$ sudo ufw allow OpenSSH
$ sudo ufw allow 'Nginx Full'   # jika error "profile not found", jalankan lagi setelah install Nginx (bab 5)
$ sudo ufw enable
$ sudo ufw status
```

Buat **swap 2 GB**. Wajib untuk RAM 1–2 GB karena proses `npm run build` boros memori:

TERMINAL · VPS

```
$ sudo fallocate -l 2G /swapfile
$ sudo chmod 600 /swapfile
$ sudo mkswap /swapfile
$ sudo swapon /swapfile
$ echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
$ free -h     # baris Swap harus 2.0Gi
```

5. Software yang di-install di VPS

Semua perintah di bab ini dijalankan sebagai user `deploy` di VPS.

5.1 Node.js 22 & PM2

TERMINAL · VPS

```
$ curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
$ sudo apt install -y nodejs git build-essential
$ node -v                # v22.x
$ sudo npm install -g pm2
```

5.2 PostgreSQL 16

TERMINAL · VPS

```
$ sudo apt install -y postgresql-common
$ sudo /usr/share/postgresql-common/pgdg/apt.postgresql.org.sh -y
$ sudo apt install -y postgresql-16
$ sudo systemctl enable --now postgresql
```

Buat database dan user khusus aplikasi. Ganti `PASSWORD_DB_KUAT` dengan password acak (hindari karakter `@ : / ? #` supaya aman di URL):

TERMINAL · VPS

```
$ sudo -u postgres psql
```

SQL · PSQL

```
CREATE USER crm WITH PASSWORD 'PASSWORD_DB_KUAT';
CREATE DATABASE crm OWNER crm;
\q
```

Postgres hanya menerima koneksi dari dalam VPS (default). Biarkan begitu dan jangan buka port 5432 di firewall.

5.3 Nginx & Certbot

TERMINAL · VPS

```
$ sudo apt install -y nginx certbot python3-certbot-nginx
$ sudo ufw allow 'Nginx Full'
$ sudo systemctl enable --now nginx
```

Buka `http://IP_VPS` di browser; halaman "Welcome to nginx!" berarti berhasil.

6. Ambil kode dari GitHub ke VPS

Karena repo private, VPS butuh izin baca. Pakai **Deploy key**: kunci yang hanya bisa membaca satu repo ini.

Di VPS:

TERMINAL · VPS

```
$ ssh-keygen -t ed25519 -C "vps-crm-deploy" -f ~/.ssh/github_crm -N ""
$ cat ~/.ssh/github_crm.pub
```

Salin seluruh baris yang muncul (diawali `ssh-ed25519`).

Di GitHub: buka repo → **Settings** → **Deploy keys** → **Add deploy key**.

- Title: `VPS produksi`
- Key: tempel baris tadi
- **Jangan** centang "Allow write access"
- Klik **Add key**.

Kembali ke VPS, beri tahu SSH agar memakai key ini untuk GitHub:

TERMINAL · VPS

```
$ cat >> ~/.ssh/config <<'EOF'
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/github_crm
  IdentitiesOnly yes
EOF
$ chmod 600 ~/.ssh/config
$ ssh -T git@github.com      # ketik yes; muncul "Hi USERNAME/crm! You've successfully authenticated"
```

Clone ke `/var/www/crm`:

TERMINAL · VPS

```
$ sudo mkdir -p /var/www/crm
$ sudo chown deploy:deploy /var/www/crm
$ git clone git@github.com:USERNAME/crm.git /var/www/crm
$ cd /var/www/crm
```

7. Konfigurasi `.env` produksi

Buat kunci rahasia untuk login:

TERMINAL · VPS

```
$ openssl rand -base64 32
```

Salin hasilnya, lalu buat file `.env`:

TERMINAL · VPS

```
$ cd /var/www/crm
$ cp .env.example .env
$ nano .env
```

Isi bagian atas menjadi seperti ini (sesuaikan):

TERMINAL · VPS

```
DATABASE_URL="postgresql://crm:PASSWORD_DB_KUAT@localhost:5432/crm?schema=public"
DATABASE_POOL_MAX=10

AUTH_SECRET="HASIL_OPENSSL_TADI"
AUTH_URL="https://crm.domainanda.com"
AUTH_TRUST_HOST=true
```

Simpan, lalu kunci aksesnya:

TERMINAL · VPS

```
$ chmod 600 .env
```

Perhatian: `AUTH_URL` harus persis sama dengan alamat yang dibuka pengguna, termasuk `https://`. Jika salah, login akan gagal atau terus kembali ke halaman login.

8. Menjalankan pertama kali

TERMINAL · VPS

```
$ cd /var/www/crm
$ npm ci                      # install dependency (+ generate Prisma client)
$ npx prisma migrate deploy   # buat semua tabel di database
$ npm run db:seed             # OPSIONAL: isi akun & data contoh
$ npm run build               # 2–5 menit
$ pm2 start ecosystem.config.cjs
$ pm2 save
$ pm2 startup                 # salin & jalankan perintah "sudo env PATH=..." yang dicetaknya
```

Cek aplikasi sudah hidup:

TERMINAL · VPS

```
$ pm2 status                          # crm harus "online"
$ curl -I http://127.0.0.1:3000/login  # harus HTTP/1.1 200
```

Perhatian, ganti password akun contoh. Jika menjalankan `db:seed`, semua akun memakai `password123`. Ganti segera, minimal untuk admin:

TERMINAL · VPS

```
$ node scripts/set-password.mjs admin@crm.local "PasswordBaruYangKuat!"
$ node scripts/set-password.mjs rina@crm.local "PasswordLainYangKuat!"
```

Disarankan tetap menjalankan `db:seed`. Selain akun contoh, seed membuat tahap pipeline (Lead → Won/Lost), board "Follow-up Sales", alasan Lost dan sumber lead. Halaman Pengaturan masih baca-saja, jadi data dasar ini belum bisa dibuat dari aplikasi.

Untuk akun tim Anda sendiri:

TERMINAL · VPS

```
$ node scripts/create-user.mjs nama@perusahaan.com "Nama Lengkap" MEMBER "PasswordKuat123!"
```

Peran bisa `ADMIN`, `MANAGER`, atau `MEMBER`. Deal contoh bisa dihapus dari drawer deal oleh Admin/Manager.

9. Domain & HTTPS

9.1 Arahkan domain ke VPS

Di panel DNS penyedia domain Anda (Niagahoster, Cloudflare, dll), tambahkan record:

Type	Name / Host	Value	TTL
A	`crm` (untuk crm.domainanda.com) atau `@` (untuk domain utama)	`IP_VPS`	Auto / 300

Tunggu 5–30 menit, lalu cek dari laptop:

TERMINAL · MAC

```
% nslookup crm.domainanda.com    # harus menampilkan IP_VPS
```

> Jika memakai Cloudflare, set awan menjadi **abu-abu (DNS only)** dulu sampai sertifikat HTTPS berhasil dibuat.

9.2 Konfigurasi Nginx

TERMINAL · VPS

```
$ sudo cp /var/www/crm/deploy/nginx-crm.conf /etc/nginx/sites-available/crm
$ sudo nano /etc/nginx/sites-available/crm      # ganti crm.domainanda.com dengan domain Anda
$ sudo ln -s /etc/nginx/sites-available/crm /etc/nginx/sites-enabled/crm
$ sudo rm -f /etc/nginx/sites-enabled/default
$ sudo nginx -t                                  # harus "syntax is ok"
$ sudo systemctl reload nginx
```

Buka `http://crm.domainanda.com` dan halaman login CRM akan muncul.

9.3 Pasang HTTPS gratis

TERMINAL · VPS

```
$ sudo certbot --nginx -d crm.domainanda.com
```

Isi email, setujui syarat, dan pilih redirect HTTP ke HTTPS jika ditanya. Sertifikat diperpanjang otomatis; cek dengan:

TERMINAL · VPS

```
$ sudo certbot renew --dry-run
```

Buka **https://crm.domainanda.com** lalu login. 🎉

10. Deploy otomatis dengan GitHub Actions

File yang sudah disiapkan di proyek:

File	Fungsi
`.github/workflows/deploy.yml`	Dijalankan GitHub setiap push ke `main`
`scripts/deploy.sh`	Dijalankan di VPS: ambil kode, install, migrasi, build, restart
`ecosystem.config.cjs`	Konfigurasi PM2

Alurnya: **push → GitHub cek typecheck & lint → jika lolos, GitHub SSH ke VPS → jalankan** `scripts/deploy.sh`. Jika pengecekan gagal, deploy dibatalkan sehingga aplikasi di VPS tetap memakai versi lama yang sehat.

10.1 Buat key khusus untuk GitHub Actions

Di VPS (user `deploy`):

TERMINAL · VPS

```
$ ssh-keygen -t ed25519 -C "github-actions" -f ~/.ssh/actions_deploy -N ""
$ cat ~/.ssh/actions_deploy.pub >> ~/.ssh/authorized_keys
$ cat ~/.ssh/actions_deploy        # ini KUNCI PRIVAT, salin seluruhnya
```

Salin semuanya, mulai dari `-----BEGIN OPENSSH PRIVATE KEY-----` sampai `-----END OPENSSH PRIVATE KEY-----`.

10.2 Isi Secrets di GitHub

Repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**. Buat 4 secret:

Name	Value
`VPS_HOST`	IP VPS, mis. `103.12.34.56`
`VPS_USER`	`deploy`
`VPS_PORT`	`22` (atau port SSH Anda jika diganti)
`VPS_SSH_KEY`	Kunci privat dari langkah 10.1

> **Perhatian:** kunci privat hanya ditempel di GitHub Secrets. Jangan ditaruh di file proyek, chat, atau email. Setelah tersimpan, hapus dari VPS jika mau: `rm ~/.ssh/actions_deploy` (file `.pub`-nya tetap ada di authorized_keys).

10.3 Uji coba

Di laptop, ubah sesuatu yang kecil (mis. teks di README), lalu:

TERMINAL · MAC

```
% git add .
% git commit -m "Uji deploy otomatis"
% git push
```

Buka repo → tab **Actions**. Job `check` lalu `deploy` akan berjalan (sekitar 3–6 menit). Tanda ✅ hijau berarti aplikasi di VPS sudah versi terbaru. Anda juga bisa menjalankan deploy manual dari tab Actions → **Deploy** → **Run workflow**.

11. Perawatan

11.1 Perintah harian

TERMINAL · VPS

```
$ pm2 status              # status aplikasi
$ pm2 logs crm            # lihat log (Control+C untuk keluar)
$ pm2 restart crm         # restart
$ pm2 monit               # pantau CPU/RAM
$ bash /var/www/crm/scripts/deploy.sh   # update manual tanpa GitHub Actions
```

Agar file log tidak memenuhi disk:

TERMINAL · VPS

```
$ pm2 install pm2-logrotate
```

11.2 Backup database harian

TERMINAL · VPS

```
$ mkdir -p ~/backups
$ crontab -e      # pilih nano jika ditanya
```

Tambahkan baris ini paling bawah (backup jam 02.00, simpan 7 hari terakhir):

```
0 2 * * * pg_dump "postgresql://crm:PASSWORD_DB_KUAT@localhost:5432/crm" | gzip > ~/backups/crm-$(date +\%F).sql.gz && find ~/backups -name 'crm-*.sql.gz' -mtime +7 -delete
```

Sesekali unduh backup ke laptop (simpan di luar VPS):

TERMINAL · MAC

```
% scp deploy@IP_VPS:~/backups/crm-2026-10-31.sql.gz .
```

Restore (mengembalikan data dari backup; data saat ini akan tertimpa):

TERMINAL · VPS

```
$ pm2 stop crm
$ sudo -u postgres psql -c "DROP DATABASE crm;" -c "CREATE DATABASE crm OWNER crm;"
$ gunzip -c ~/backups/crm-2026-10-31.sql.gz | psql "postgresql://crm:PASSWORD_DB_KUAT@localhost:5432/crm"
$ pm2 start crm
```

11.3 Update keamanan server

Jalankan sebulan sekali:

TERMINAL · VPS

```
$ sudo apt update && sudo apt upgrade -y
$ sudo reboot      # jika diminta; PM2 akan menyalakan aplikasi lagi otomatis
```

12. Troubleshooting

Gejala	Penyebab umum	Solusi
502 Bad Gateway	Aplikasi mati / belum di-build	`pm2 status`, `pm2 logs crm`. Jalankan ulang `bash scripts/deploy.sh`
Build berhenti: **JavaScript heap out of memory** / "Killed"	RAM kurang	Pastikan swap aktif (`free -h`), lihat bab 4.5
Login gagal, kembali ke halaman login terus	`AUTH_URL` tidak sama dengan alamat di browser	Perbaiki `.env`, lalu `pm2 restart crm --update-env`
Error **"MissingSecret"** di log	`AUTH_SECRET` kosong	Isi di `.env`, lalu restart
Can't reach database server / P1001	Postgres mati atau `DATABASE_URL` salah	`sudo systemctl status postgresql`; cek user/password di `.env`
password authentication failed	Password DB di `.env` beda	Samakan, atau reset: `sudo -u postgres psql -c "ALTER USER crm WITH PASSWORD '...';"`
Actions gagal: **ssh: handshake failed** / unable to authenticate	Secret salah / key tidak lengkap	Tempel ulang `VPS_SSH_KEY` lengkap dengan baris BEGIN/END; cek `VPS_USER`, `VPS_PORT`
Actions gagal di `git fetch` (di VPS)	Deploy key belum dipasang	Ulangi bab 6; tes `ssh -T git@github.com` di VPS
Actions gagal di job **check**	Ada error typecheck/lint	Jalankan `npm run typecheck` dan `npm run lint` di laptop, perbaiki, push lagi
`certbot` gagal	DNS belum mengarah / Cloudflare proxy aktif	Cek `nslookup`; set Cloudflare ke DNS only; pastikan port 80 terbuka
Perubahan tidak muncul setelah deploy	Cache browser	Hard refresh `Cmd+Shift+R`; cek tab Actions benar-benar hijau

13. Checklist ringkas

Laptop

- [ ] Homebrew, Node.js 22, Git, GitHub CLI, VS Code ter-install (`node -v` , `git --version`)
- [ ] `git config` nama & email sudah diatur
- [ ] Repo **private** dibuat di GitHub, kode sudah di-push, `.env` **tidak** ikut

VPS

- [ ] User `deploy` + login SSH key, root & password login dimatikan
- [ ] Firewall aktif (OpenSSH, Nginx Full), swap 2 GB
- [ ] Node 22, PM2, PostgreSQL 16, Nginx, Certbot ter-install
- [ ] Database `crm` + user `crm` dibuat
- [ ] Deploy key dipasang, repo di-clone ke `/var/www/crm`
- [ ] `.env` produksi diisi (`DATABASE_URL` , `AUTH_SECRET` , `AUTH_URL=https://…`)
- [ ] `npm ci` → `prisma migrate deploy` → `build` → `pm2 start` → `pm2 save` → `pm2 startup`
- [ ] Password akun contoh sudah diganti

Domain & Otomasi

- [ ] Record A domain → IP VPS, Nginx dikonfigurasi, HTTPS aktif via Certbot
- [ ] 4 secrets GitHub (`VPS_HOST` , `VPS_USER` , `VPS_PORT` , `VPS_SSH_KEY`) diisi
- [ ] Uji `git push` → tab Actions hijau → perubahan tampil di website
- [ ] Backup harian via cron aktif